

Φάση 3 – Υβριδικό Σύστημα Ανάκτησης

Σε αυτή την φάση θα υλοποιήσουμε ένα υβριδικό σύστημα ανάκτησης πληροφορίας πάνω στην συλλογή IR2025, το οποίο συνδυάζει την κλασική ανάκτηση κειμένων με BM25 similarity μέσω Elasticsearch και την σημασιολογική ανάκτηση χρησιμοποιώντας embeddings και FAISS.

Με την προσέγγιση αυτή αξιοποιούμε την ταχύτητα της κλασικής ανάκτησης για τον εντοπισμό ενός αρχικού συνόλου κειμένων καθώς και την σημασιολογική ικανότητα των Transformers για επαναταξινόμηση των αποτελεσμάτων.

1. Για κάθε ερώτημα της συλλογής IR2025: α) Εκτελέστε το ερώτημα στο ευρετήριο που υλοποιήσατε στη Φάση 1. Επιστρέψτε τα $N > k$ πιο σχετικά κείμενα (ενδεικτικά $N = 200$). β) Με το ίδιο προεκπαιδευμένο μοντέλο *transformer* της Φάσης 2, δημιουργήστε ενθέσεις (*embeddings*) για τα N κείμενα που επέστρεψε η *ElasticSearch* για το ερώτημα, καθώς και για το ίδιο το ερώτημα. γ) Δημιουργήστε δυναμικά ένα *FAISS index* με τα *embeddings* των N κειμένων. δ) Εκτελέστε το ερώτημα. Αναζητήστε στο ευρετήριο *FAISS* τα k πιο κοντινά κείμενα, για $k = 20, 30, 50$, βάσει συνημιτονοειδούς ομοιότητας (*cosine similarity*). Επιστρέψτε τα αποτελέσματα ταξινομημένα κατά φθίνουσα ομοιότητα.

Αρχικά γίνεται η σύνδεση στην Elasticsearch για να χρησιμοποιήσουμε το μοντέλο BM25 της φάσης 1.

```
!curl http://localhost:9200/

{
  "name" : "LAPTOP-UDLON1P4",
  "cluster_name" : "elasticsearch",
  "cluster_uuid" : "6eRXGDVkrZa-7IQXW6AsSg",
  "version" : {
    "number" : "9.2.3",
    "build_flavor" : "default",
    "build_type" : "zip",
    "build_hash" : "d8972a71dbbd64ff17f2f4dba9ca2c3fe09fb100",
    "build_date" : "2025-12-16T10:09:08.849001802Z",
    "build_snapshot" : false,
    "lucene_version" : "10.3.2",
    "minimum_wire_compatibility_version" : "8.19.0",
    "minimum_index_compatibility_version" : "8.0.0"
  },
  "tagline" : "You Know, for Search"
}
```

Θα χρησιμοποιήσουμε το ευρετήριο της Φάσης 1 με το BM25 Similarity για την ανάκτηση ενός αρχικού υποσυνόλου κειμένων.

```
# index from phase 1
exists = client.indices.exists(index="my_idx")
print("Index exists: ", exists)

mapp= client.indices.get_mapping(index="my_idx")
print(mapp["my_idx"]["mappings"])

Index exists: True
{'properties': {'doc_id': {'type': 'integer'}, 'text': {'type': 'text', 'analyzer': 'default', 'similarity': 'my_bm25'}}}
```

Για το δεύτερο στάδιο της υβριδικής ανάκτησης χρησιμοποιούμε το μοντέλο [all-MiniLM-L6-v2](#) όπως στην 2^η Φάση.

```
# model from phase 2
model = SentenceTransformer("all-MiniLM-L6-v2")

print(model)

SentenceTransformer(
  (0): Transformer({'max_seq_length': 256, 'do_lower_case': False, 'architecture': 'BertModel'})
  (1): Pooling({'word_embedding_dimension': 384, 'pooling_mode_cls_token': False, 'pooling_mode_mean_tokens': True, 'pooling_mode_max_tokens': False, 'pooling_mode_mean_sqrt_len_tokens': False, 'pooling_mode_weightedmean_tokens': False, 'pooling_mode_lasttoken': False, 'include_prompt': True})
  (2): Normalize()
)
```

Διαβάζουμε το αρχείο με τα ερωτήματα και αρχικοποιούμε τις παραμέτρους N για το υποσύνολο των ανακτηθέντων κειμένων και a για τα βάρη των σκορ για τον συνδυασμό τους.

```
## reading the queries file
queries = pd.read_csv("IR2025/queries.csv")
print(queries.head())
```

	ID	Text
0	Q01	EUTRAVEL Optimodal European Travel Ecosystem E...
1	Q02	Track And Know Big Data for Mobility Tracking ...
2	Q03	SELIS, Towards a Shared European Logistics Int...
3	Q04	TYPHON Polyglot and Hybrid Persistence Archite...
4	Q05	CHARIOT Cognitive Heterogeneous Architecture f...

```
[22]:

# parameters
N = 200
a = 0.3 # for score
```

Οι κύριες συναρτήσεις είναι η `run_hybrid` και η `hybrid_search`.

Η `run_hybrid` εκτελεί την υβριδική αναζήτηση για κάθε ερώτημα μέσω της `hybrid_search` και αποθηκεύει τα αποτελέσματα σε μορφή κατάλληλη για το εργαλείο `trec_eval`

Η διαδικασία της υβριδικής ανάκτησης φαίνεται στην συνάρτηση `hybrid_search`, η οποία κάνει τα εξής:

α) Εκτελέστε το ερώτημα στο ευρετήριο που υλοποιήσατε στη Φάση 1. Επιστρέψτε τα $N \gg k$ πιο σχετικά κείμενα.

Πραγματοποιεί κλασική ανάκτηση – λεκτική αναζήτηση μέσω ομοιότητας BM25 χρησιμοποιώντας το ευρετήριο της 1^{ης} Φάσης και επιστρέφει τα N=200 πιο συναφή κείμενα.

Αποθηκεύει το περιεχόμενο των κειμένων καθώς και τα ids και τα σκορ ομοιότητάς που προέκυψαν.

Κάνει κανονικοποίηση στο σκορ ομοιότητας ώστε να βρίσκεται στο εύρος [0, 1] για να το χρησιμοποιήσουμε στην συνέχεια για τον υπολογισμό της τελικής συνδυαστικής ομοιότητας.

```
# normalizing bm25 scores to be in range [0,1]
bm25_scores = np.array(bm25_scores)
bm25_scores = (bm25_scores - bm25_scores.min()) / (bm25_scores.max() - bm25_scores.min() + 1e-9)
```

β) Με το ίδιο προεκπαιδευμένο μοντέλο `transformer` της Φάσης 2, δημιουργήστε ενθέσεις (`embeddings`) για τα N κείμενα που επέστρεψε η `ElasticSearch` για το ερώτημα, καθώς και για το ίδιο το ερώτημα.

Στην συνέχεια χρησιμοποιεί το προεκπαιδευμένο μοντέλο `all-MiniLM-L6-v2` για την δημιουργία ενθέσεων των 200 κειμένων και του ερωτήματος.

Τα `embeddings` δημιουργήθηκαν όπως και στην 2^η Φάση και χρειάστηκαν περίπου 5 λεπτά για την εκτέλεση τους.

γ) Δημιουργήστε δυναμικά ένα `FAISS index` με τα `embeddings` των N κειμένων.

Δημιουργεί ευρετήριο FAISS τύπου IndexFlatIP το οποίο υλοποιεί την συνημιτονοειδή ομοιότητα όπως και στην Φάση 2.

Γίνεται εισαγωγή των embeddings χωρίς τα αντίστοιχα αναγνωριστικά τους ώστε να υλοποιήσουμε την συνδυαστική βαθμολογία και να κάνουμε την ταξινόμηση σύμφωνα με αυτήν.

δ) Εκτελέστε το ερώτημα.

Εκτελείται η αναζήτηση του ερωτήματος στο ευρετήριο, η οποία επιστρέφει τα cosine scores και μια λίστα με τις ταξινομημένες βάσει συνάφειας θέσεις των embeddings στο FAISS ευρετήριο που αντιστοιχούν στα doc ids των αποτελεσμάτων για τα 200 κείμενα.

Από αυτή την λίστα γίνεται το mapping όπου αντιστοιχίζονται οι θέσεις των αποτελεσμάτων της FAISS με τα ids των εγγράφων και τις αντίστοιχες βαθμολογίες BM25 για να υπολογίσουμε την τελική συνδυαστική βαθμολογία.

Κάνει κανονικοποίηση του cosine score για τα είναι στο εύρος [0, 1] όπως και το bm25 similarity.

Υπολογίζεται το τελικό σκορ ως σταθμισμένος μέσος όρων των BM25 score και cosine similarity με παράμετρο α, όπου α το βάρος του cosine score, δηλαδή η συνεισφορά της σημασιολογικής ομοιότητας και (1-α) το βάρος του BM25 δηλαδή η συνεισφορά της λεκτικής ομοιότητας.

Γίνεται ταξινόμηση των κειμένων βάσει του τελικού σκορ και επιστρέφει τα k πιο συναφή.

```
# σταθμισμένο μέσο όρο BM25 score και cosine similarity
final_scores = a*cosine_scores + (1-a)*selected_bm25
```

--

Αναζητήστε στο ευρετήριο FAISS τα k πιο κοντινά κείμενα, για k = 20, 30, 50, βάσει συνημιτονοειδούς ομοιότητας (cosine similarity) ή L2 απόστασης. Επιστρέψτε τα αποτελέσματα ταξινομημένα κατά φθίνουσα ομοιότητα. Η τελική κατάταξη είναι αυτή της FAISS, αλλά μπορείτε να πειραματιστείτε με συνδυασμούς βαθμολογίας

Για κάθε k πιο κοντινά κείμενα όπου k=20, 30, 50 εκτελείται η μέθοδος hybrid_run η οποία παίρνει ως είσοδο το αρχείο με όλα τα ερωτήματα. Για καθένα από τα ερωτήματα βρίσκει το id και το περιεχόμενό του και καλεί την συνάρτηση hybrid_search για να πάρει τα k πιο σχετικά έγγραφα και τα σκορ ομοιότητάς τους όπως υπολογίστηκαν συνδυαστικά. Έπειτα, αποθηκεύει το κάθε αποτέλεσμα σε λίστα με μορφή κατάλληλη για το εργαλείο αξιολόγησης trec_eval.

```
# running for k = 20 , 30 , 50 documents
results_20 = run_hybrid(queries, k=20)
print(results_20)
results_30 = run_hybrid(queries, k=30)
results_50 = run_hybrid(queries, k=50)
```

Batches: 100%		7/7	[00:15<00:00, 2.25s/it]
Batches: 100%		7/7	[00:15<00:00, 2.24s/it]
Batches: 100%		7/7	[00:16<00:00, 2.29s/it]
Batches: 100%		7/7	[00:16<00:00, 2.37s/it]
Batches: 100%		7/7	[00:16<00:00, 2.36s/it]
Batches: 100%		7/7	[00:16<00:00, 2.32s/it]

```
[{'query_id': 'Q01', 'doc_id': 193378, 'rank': 1, 'score': 1.0000000119198482}, {'query_id': 'Q01', 'doc_id': 193373, 'rank': 2, 'score': 0.3578958753608174}, {'query_id': 'Q01', 'doc_id': 193715, 'rank': 3, 'score': 0.3210656597166335}, {'query_id': 'Q01', 'doc_id': 205685, 'rank': 4, 'score': 0.306915808957431}, {'query_id': 'Q01', 'doc_id': 206230, 'rank': 5, 'score': 0.3045921282922276}, {'query_id': 'Q01', 'doc_id': 193375, 'rank': 6, 'score': 0.29722598494913616}, {'query_id': 'Q01', 'doc_id': 210137, 'rank': 7, 'score': 0.2958429444537096}, {'query_id': 'Q01', 'doc_id': 211346, 'rank': 8, 'score': 0.28332106845286503}, {'query_id': 'Q01', 'doc_id': 206228, 'rank': 9, 'score': 0.28273256542968167}, {'query_id': 'Q01', 'doc_id': 193353, 'rank': 10, 'score': 0.26677322444052776}, {'query_id': 'Q01', 'doc_id': 198900, 'rank': 11, 'score': 0.26295340313082116}, {'query_id':
```

Στην συνέχεια, οι λίστες με τα αποτελέσματα αποθηκεύονται σε αρχεία txt για να χρησιμοποιηθούν στο trec_eval, με την παρακάτω μορφή.

```
with open("trec_eval/run_hybrid_k20.txt") as file:
    print(file.read())
```

```
Q01 Q0 193378 1 1.0000000119198482 hybrid_k20
Q01 Q0 193373 2 0.35789587453608174 hybrid_k20
Q01 Q0 193715 3 0.3210656597166335 hybrid_k20
Q01 Q0 205685 4 0.306915808957431 hybrid_k20
Q01 Q0 206230 5 0.3045921282922276 hybrid_k20
Q01 Q0 193375 6 0.29722598494913616 hybrid_k20
Q01 Q0 210137 7 0.2958429444537096 hybrid_k20
Q01 Q0 211346 8 0.28332106845286503 hybrid_k20
Q01 Q0 206228 9 0.28273256542968167 hybrid_k20
Q01 Q0 193353 10 0.26677322444052776 hybrid_k20
Q01 Q0 198900 11 0.26295340313082116 hybrid_k20
Q01 Q0 202703 12 0.2604626610292017 hybrid_k20
Q01 Q0 211970 13 0.26031124523978727 hvbrid k20
```

2. Αξιολογήστε τα αποτελέσματά σας συγκρίνοντάς τα με τις σωστές απαντήσεις χρησιμοποιώντας το εργαλείο αξιολόγησης trec_eval και τα μέτρα αξιολόγησης MAP (mean average precision) και Precision@k (μέση ακρίβεια στα k πρώτα ανακτηθέντα κείμενα) για k = 5, 10, 15, 20.

Μετακινούμε τα 3 αρχεία που δημιουργήθηκαν στον φάκελο με το εργαλείο trec_eval.

Αρχικά εκτελούμε την εντολή >trec_eval -q qrels.txt run_hybrid_k20.txt για να πάρουμε τα αποτελέσματα της αξιολόγησης στα **k=20 κείμενα**.

Μετά >trec_eval -q qrels.txt run_hybrid_k30.txt για **k=30 κείμενα** και >trec_eval -q qrels.txt run_hybrid_k50.txt για **k=50 κείμενα**.

Δοκιμάστηκαν διάφορες τιμές της παραμέτρου α (0.7, 0.5, 0.3) για την τελική βαθμολογία που επηρεάζει την συμβολή της σημασιολογικής ανάκτησης και της κλασικής.

Συγκεκριμένα για α=0.7 με περισσότερο βάρος στην σημασιολογική αναζήτηση βγήκε το παρακάτω αποτέλεσμα:

map	all	0.4640
gm_map	all	0.4543
Rprec	all	0.5297
bpref	all	0.6137
recip_rank	all	0.9500
iprec_at_recall_0.00	all	0.9833
iprec_at_recall_0.10	all	0.9583
iprec_at_recall_0.20	all	0.8850
iprec_at_recall_0.30	all	0.7653
iprec_at_recall_0.40	all	0.6752
iprec_at_recall_0.50	all	0.6184
iprec_at_recall_0.60	all	0.3481
iprec_at_recall_0.70	all	0.0833
iprec_at_recall_0.80	all	0.0000
iprec_at_recall_0.90	all	0.0000
iprec_at_recall_1.00	all	0.0000
P_5	all	0.7400
P_10	all	0.6500
P_15	all	0.5333
P_20	all	0.4700
P_30	all	0.3133
P_100	all	0.0940
P_200	all	0.0470
P_500	all	0.0188
P_1000	all	0.0094

Εικόνα 1: k=20

map	all	0.5287
gm_map	all	0.5207
Rprec	all	0.5297
bpref	all	0.7679
recip_rank	all	0.9500
iprec_at_recall_0.00	all	0.9833
iprec_at_recall_0.10	all	0.9583
iprec_at_recall_0.20	all	0.8850
iprec_at_recall_0.30	all	0.7653
iprec_at_recall_0.40	all	0.6752
iprec_at_recall_0.50	all	0.6196
iprec_at_recall_0.60	all	0.5185
iprec_at_recall_0.70	all	0.3966
iprec_at_recall_0.80	all	0.1695
iprec_at_recall_0.90	all	0.0333
iprec_at_recall_1.00	all	0.0000
P_5	all	0.7400
P_10	all	0.6500
P_15	all	0.5333
P_20	all	0.4700
P_30	all	0.3867
P_100	all	0.1160
P_200	all	0.0580
P_500	all	0.0232
P_1000	all	0.0116

Εικόνα 2: k=30

map	all	0.5778
gm_map	all	0.5669
Rprec	all	0.5297
bpref	all	0.9003
recip_rank	all	0.9500
iprec_at_recall_0.00	all	0.9833
iprec_at_recall_0.10	all	0.9583
iprec_at_recall_0.20	all	0.8850
iprec_at_recall_0.30	all	0.7653
iprec_at_recall_0.40	all	0.6752
iprec_at_recall_0.50	all	0.6196
iprec_at_recall_0.60	all	0.5185
iprec_at_recall_0.70	all	0.4244
iprec_at_recall_0.80	all	0.3302
iprec_at_recall_0.90	all	0.2524
iprec_at_recall_1.00	all	0.0636
P_5	all	0.7400
P_10	all	0.6500
P_15	all	0.5333
P_20	all	0.4700
P_30	all	0.3867
P_100	all	0.1380
P_200	all	0.0690
P_500	all	0.0276
P_1000	all	0.0138

Εικόνα 3: k=50

Στην δεύτερη περίπτωση με α=0.5 όπου και οι δύο τρόποι ανάκτησης έχουν την ίδια βαρύτητα βγήκε αποτέλεσμα:

map	all	0.5684
gm_map	all	0.5656
Rprec	all	0.5969
bpref	all	0.7143
recip_rank	all	0.9500
iprec_at_recall_0.00	all	0.9875
iprec_at_recall_0.10	all	0.9708
iprec_at_recall_0.20	all	0.9508
iprec_at_recall_0.30	all	0.8384
iprec_at_recall_0.40	all	0.7780
iprec_at_recall_0.50	all	0.7479
iprec_at_recall_0.60	all	0.6596
iprec_at_recall_0.70	all	0.3672
iprec_at_recall_0.80	all	0.1121
iprec_at_recall_0.90	all	0.0000
iprec_at_recall_1.00	all	0.0000
P_5	all	0.8200
P_10	all	0.7100
P_15	all	0.6400
P_20	all	0.5400
P_30	all	0.3600
P_100	all	0.1080
P_200	all	0.0540
P_500	all	0.0216
P_1000	all	0.0108

Εικόνα 1: k=20

map	all	0.6280
gm_map	all	0.6239
Rprec	all	0.5969
bpref	all	0.8379
recip_rank	all	0.9500
iprec_at_recall_0.00	all	0.9875
iprec_at_recall_0.10	all	0.9708
iprec_at_recall_0.20	all	0.9508
iprec_at_recall_0.30	all	0.8384
iprec_at_recall_0.40	all	0.7780
iprec_at_recall_0.50	all	0.7479
iprec_at_recall_0.60	all	0.6596
iprec_at_recall_0.70	all	0.5663
iprec_at_recall_0.80	all	0.3184
iprec_at_recall_0.90	all	0.0875
iprec_at_recall_1.00	all	0.0370
P_5	all	0.8200
P_10	all	0.7100
P_15	all	0.6400
P_20	all	0.5400
P_30	all	0.4233
P_100	all	0.1270
P_200	all	0.0635
P_500	all	0.0254
P_1000	all	0.0127

Εικόνα 2: k=30

map	all	0.6689
gm_map	all	0.6630
Rprec	all	0.5969
bpref	all	0.9411
recip_rank	all	0.9500
iprec_at_recall_0.00	all	0.9875
iprec_at_recall_0.10	all	0.9708
iprec_at_recall_0.20	all	0.9508
iprec_at_recall_0.30	all	0.8384
iprec_at_recall_0.40	all	0.7780
iprec_at_recall_0.50	all	0.7479
iprec_at_recall_0.60	all	0.6596
iprec_at_recall_0.70	all	0.5663
iprec_at_recall_0.80	all	0.4235
iprec_at_recall_0.90	all	0.3715
iprec_at_recall_1.00	all	0.1686
P_5	all	0.8200
P_10	all	0.7100
P_15	all	0.6400
P_20	all	0.5400
P_30	all	0.4233
P_100	all	0.1440
P_200	all	0.0720
P_500	all	0.0288
P_1000	all	0.0144

Εικόνα 3: k=50

Στην τρίτη περίπτωση με $\alpha=0.3$ με περισσότερο βάρος στην κλασική ανάκτηση με λεκτική αναζήτηση βγήκε αποτέλεσμα:

map	all	0.6698
gm_map	all	0.6685
Rprec	all	0.6573
bpref	all	0.7901
recip_rank	all	0.9500
iprec_at_recall_0.00	all	0.9857
iprec_at_recall_0.10	all	0.9857
iprec_at_recall_0.20	all	0.9857
iprec_at_recall_0.30	all	0.9325
iprec_at_recall_0.40	all	0.8740
iprec_at_recall_0.50	all	0.8558
iprec_at_recall_0.60	all	0.7597
iprec_at_recall_0.70	all	0.6245
iprec_at_recall_0.80	all	0.2550
iprec_at_recall_0.90	all	0.0600
iprec_at_recall_1.00	all	0.0000
P_5	all	0.9000
P_10	all	0.7800
P_15	all	0.6800
P_20	all	0.5950
P_30	all	0.3967
P_100	all	0.1190
P_200	all	0.0595
P_500	all	0.0238
P_1000	all	0.0119

Εικόνα 1: k=20

map	all	0.7411
gm_map	all	0.7375
Rprec	all	0.6621
bpref	all	0.9086
recip_rank	all	0.9500
iprec_at_recall_0.00	all	0.9857
iprec_at_recall_0.10	all	0.9857
iprec_at_recall_0.20	all	0.9857
iprec_at_recall_0.30	all	0.9325
iprec_at_recall_0.40	all	0.8740
iprec_at_recall_0.50	all	0.8558
iprec_at_recall_0.60	all	0.7597
iprec_at_recall_0.70	all	0.6972
iprec_at_recall_0.80	all	0.5119
iprec_at_recall_0.90	all	0.3676
iprec_at_recall_1.00	all	0.1611
P_5	all	0.9000
P_10	all	0.7800
P_15	all	0.6800
P_20	all	0.5950
P_30	all	0.4633
P_100	all	0.1390
P_200	all	0.0695
P_500	all	0.0278
P_1000	all	0.0139

Εικόνα 2: k=30

map	all	0.7660
gm_map	all	0.7628
Rprec	all	0.6621
bpref	all	0.9762
recip_rank	all	0.9500
iprec_at_recall_0.00	all	0.9857
iprec_at_recall_0.10	all	0.9857
iprec_at_recall_0.20	all	0.9857
iprec_at_recall_0.30	all	0.9325
iprec_at_recall_0.40	all	0.8740
iprec_at_recall_0.50	all	0.8558
iprec_at_recall_0.60	all	0.7597
iprec_at_recall_0.70	all	0.6972
iprec_at_recall_0.80	all	0.5728
iprec_at_recall_0.90	all	0.4812
iprec_at_recall_1.00	all	0.3688
P_5	all	0.9000
P_10	all	0.7800
P_15	all	0.6800
P_20	all	0.5950
P_30	all	0.4633
P_100	all	0.1490
P_200	all	0.0745
P_500	all	0.0298
P_1000	all	0.0149

Εικόνα 3: k=50

Από τα αποτελέσματα έχουμε:

α	MAP@20	MAP@30	MAP@50
0.7	0.4640	0.5287	0.5778
0.5	0.5684	0.6280	0.6689
0.3	0.6698	0.7411	0.7660

Παρατηρούμε ότι όσο μειώνεται η τιμή του α αυξάνεται το MAP, δηλαδή όσο μειώνεται το βάρος του cosine similarity της σημασιολογικής αναζήτησης αυξάνεται η απόδοση. Η καλύτερη απόδοση σημειώθηκε για $\alpha=0.3$ κάτι που υποδεικνύει ότι η λεκτική αναζήτηση με BM25 ταιριάζει περισσότερο στην συλλογή κειμένων IR2025 με τα δοθέντα ερωτήματα σε σχέση με την σημασιολογική ομοιότητα.

Θα συνεχίσουμε τον σχολιασμό για τα αποτελέσματα με τιμή $\alpha=0.3$.

Καταλήγουμε σε αυτόν τον πίνακα αποτελεσμάτων:

k	20	30	50
map	0.6698	0.7411	0.7660
P_5	0.9000	0.9000	0.9000
P_10	0.7800	0.7800	0.7800
P_15	0.6800	0.6800	0.6800
P_20	0.5950	0.5950	0.5950
P_30	0.3967	0.4633	0.4633
P_100	0.1190	0.1390	0.1490
P_200	0.0595	0.0695	0.0745
P_500	0.0238	0.0278	0.0298
P_1000	0.0119	0.0139	0.0149

Παρατηρούμε ότι το MAP (mean average precision) αυξάνεται καθώς αυξάνεται το πλήθος των ανακτηθέντων κειμένων k . Συγκεκριμένα το map αυξάνεται από 0.6698 για $k=20$, σε 0.7411 για $k=30$ και σε 0.7660 για $k=50$. Η μεγαλύτερη αύξηση εμφανίζεται από τα 20 στα 30 κείμενα ($0.7411 - 0.6698 = 0.0723$), ενώ από τα 30 στα 50 κείμενα η βελτίωση είναι μικρότερη ($0.7660 - 0.7411 = 0.0249$).

Βλέπουμε ότι για σταθερό πλήθος ανακτηθέντων κειμένων η ακρίβεια (precision) μειώνεται όσο αυξάνεται το k , που είναι λογικό αφού μπαίνουν όλο και περισσότερα μη συναφή κείμενα στην ανάκτηση. Το Precision@ k για k μέχρι 20 κείμενα παραμένει σταθερό για τα πρώτα 20,30 και 50 ανακτηθέντα κείμενα, συμπεραίνοντας ότι το σύστημα ανακτά τα συναφή έγγραφα νωρίς και ότι η κατάταξη των ανακτηθέντων εγγράφων είναι σταθερή για τα 20 πρώτα.

Τελικά, μπορούμε να συμπεράνουμε ότι το σύστημα ανάκτησης που φτιάξαμε πετυχαίνει υψηλή ακρίβεια, με MAP γύρω στο 70% στα πρώτα αποτελέσματα. Οι τιμές του Precision@5 μέχρι Precision@20 είναι σταθερές και ανεξάρτητες του πλήθους των ανακτηθέντων κειμένων, κάτι που δείχνει ότι η κατάταξη στα κορυφαία αποτελέσματα είναι σταθερή και άρα αξιόπιστη. Γενικά, η αύξηση που MAP αν και φθίνουσα υποδεικνύει ότι το σύστημα συνεχίζει να ανακτά συναφή κείμενα και σε χαμηλότερη κατάταξη.

3. Καταγράψτε τα πειράματά σας σε μια αναφορά όπως στις προηγούμενες φάσεις, σχόλια για πλεονεκτήματα/μειονεκτήματα της υβριδικής προσέγγισης,. Στην αναφορά σας συγκρίνετε τα αποτελέσματα της Φάσης 3 με τα αποτελέσματα των προηγούμενων φάσεων.

Συνοπτικά το υβριδικό σύστημα λειτουργεί ως εξής:

Για κάθε ερώτημα αρχικά εκτελείται αναζήτηση στο ευρετήριο της 1^{ης} Φάσης μέσω Elasticsearch και ανακτώνται τα $N=200$ πιο σχετικά κείμενα σύμφωνα με το BM25 score ομοιότητας. Στην συνέχεια, δημιουργούνται embeddings για τα 200 αυτά κείμενα και το ερώτημα χρησιμοποιώντας το προεκπαιδευμένο μοντέλο της 2^{ης} Φάσης.

Επιλέγουμε $N=200$ κείμενα για την αρχική ανάκτηση ώστε να πετύχουμε ισορροπία μεταξύ αποτελεσματικότητας και υπολογιστικού κόστους. Συγκεκριμένα, με την χρήση ενός σχετικά μεγάλου N αυξάνεται η πιθανότητα τα υποψήφια ανακτηθέντα κείμενα να είναι όντως συναφή, επιτρέποντας στην συνέχεια στην σημασιολογικά επανακατάταξη μέσω FAISS να βελτιώσει την τελική ακρίβεια. Αντίθετα, η επιλογή ενός υπερβολικά μεγάλου N θα αύξανε σημαντικά το υπολογιστικό κόστος δημιουργίας των embeddings και αναζήτησης στο ευρετήριο FAISS.

Τα embeddings των 200 κειμένων εισάγονται σε ένα προσωρινό ευρετήριο FAISS στο οποίο εκτελείται αναζήτηση k κοντινότερων γειτόνων βάσει του cosine score ομοιότητας. Η τελική κατάταξη των k κειμένων προκύπτει από τον σταθμισμένο μέσο όρο BM25 score και cosine similarity.

Αυτό το υβριδικό σύστημα είναι αποτελεσματικό καθώς συνδυάζει κλασική και σημασιολογική ανάκτηση. Μειώνεται σημαντικά ο υπολογιστικός φόρτος αφού αρχικά εκτελείται κλασική ανάκτηση, ενώ η σημασιολογική

σύγκριση δεν πραγματοποιείται σε ολόκληρη την συλλογή αλλά σε ένα μικρό υποσύνολο των 200 κειμένων. Τα αποτελέσματα είναι καλύτερα αφού πραγματοποιείται σημασιολογική επαναταξινόμηση και όχι αποκλειστικά αναζήτηση βάσει λεκτικής ομοιότητας.

Με αυτή την προσέγγιση το σύστημα μπορεί να χειριστεί τόσο ερωτήματα με μεγάλη λεκτική αντιστοίχιση όσο και σημασιολογικά συναφή. Με την παράμετρο α της συνάρτησης βαθμολογίας μπορεί να ρυθμιστεί το βάρος της συνεισφοράς της κάθε μεθόδου, δηλαδή να προσαρμόζεται ανάλογα με την συλλογή κειμένων.

Από την άλλη, το υβριδικό σύστημα έχει μεγαλύτερη υπολογιστική πολυπλοκότητα σε σχέση με την κλασική ανάκτηση, αφού χρειάζεται επιπλέον χρόνο και μνήμη για την δημιουργία των embeddings και του δυναμικού ευρετηρίου FAISS για κάθε ερώτημα. Γενικά η απόδοσή του εξαρτάται σε μεγάλο βαθμό από την επιλογή των παραμέτρων N και α σε συνδυασμό με την δεδομένη συλλογή κειμένων και ερωτημάτων.

ΣΥΓΚΡΙΣΗ

Τα αποτελέσματα των 3 συστημάτων είναι τα ακόλουθα:

	MAP@20	MAP@30	MAP@50
Classic	0.6895	0.7471	0.7701
Sematic	0.3129	0.3413	0.3744
Hybrid	0.6698	0.7411	0.7660

Το σύστημα της κλασικής ανάκτησης BM25 έχει την υψηλότερη συνολική απόδοση σε όλες τις τιμές του k .

Το υβριδικό σύστημα εμφανίζει ελάχιστα χειρότερα αποτελέσματα έχοντας χρησιμοποιήσει την παράμετρο $\alpha=0.3$ που δίνει μεγαλύτερο βάρος στο BM25 similarity score. Αυτό δείχνει ότι η συλλογή κειμένων IR2025 ευνοεί την λεκτική σύγκριση ομοιότητας μεταξύ ερωτημάτων και συναφών κειμένων, που κάνει τις κλασικές μεθόδους ανάκτησης πιο αποδοτικές.

Γενικά, όσον αφορά τα δεδομένα μας, τα ερωτήματα είναι περιγραφικά και μεγάλα σε έκταση, περιλαμβάνοντας εξειδικευμένες λέξεις που εμφανίζονται αυτούσιες στα συναφή κείμενα. Αυτό επιφέρει καλύτερα αποτελέσματα με την κλασική ανάκτηση που κάνει αντιστοίχιση όρων. Αντίθετα, η σημασιολογική ανάκτηση μπορεί να οδηγήσει στην ανάκτηση κειμένων τα οποία είναι θεματικά συναφή αλλά δεν χαρακτηρίζονται σχετικά σύμφωνα με τα αποτελέσματα qrels. Επιπλέον, το μοντέλο από τη βιβλιοθήκη transformers που χρησιμοποιήσαμε στην σημασιολογική ανάκτηση είναι προεκπαιδευμένο, δηλαδή δεν έχει εκπαιδευτεί ειδικά για την συγκεκριμένη συλλογή κειμένων μας ώστε να επιφέρει καλύτερα αποτελέσματα.

Συγκριτικά με το σύστημα σημασιολογικής ανάκτησης, που έχει ακρίβεια γύρω στο 35%, το υβριδικό σύστημα είναι σαφώς καλύτερο, κάτι που φαίνεται από την σημαντική βελτίωση της ακρίβειας. Αυτό συμβαίνει γιατί η αρχική ανάκτηση στο υβριδικό σύστημα γίνεται μέσω της Elasticsearch και λειτουργεί ως φίλτρο, μειώνοντας σημαντικά τον αριθμό των κειμένων που θα εξεταστούν στην συνέχεια για την σημασιολογική τους ομοιότητα. Με αυτό τον τρόπο η σημασιολογική σύγκριση γίνεται σε ήδη σχετικά κείμενα. Η δραματική βελτίωση της απόδοσης του υβριδικού συστήματος όμως αποδίδεται σε μεγάλο βαθμό στην βαρύτητα που είχε το BM25 σκορ ομοιότητας έναντι του cosine similarity score στην τελική συνδυαστική βαθμολογία.

Τελικά, τα αποτελέσματα δείχνουν ότι το υβριδικό σύστημα αποτελεί μια ισορροπημένη προσέγγιση, η οποία συνδυάζει τα πλεονεκτήματα τόσο της κλασικής όσο και της σημασιολογικής ανάκτησης. Παρόλο που δεν πέτυχε την υψηλότερη συνολική απόδοση, κατάφερε σημαντική βελτίωση σε σχέση με την σημασιολογική ανάκτηση και παρουσίασε μεγαλύτερη ευελιξία ως προς την διαχείριση διαφορετικών τύπων ερωτημάτων.